AI-Driven Stock Market Simulation and Trading App

Andrew Fagan

Quick Description of the Project

- This capstone project aimed to develop a comprehensive application for tracking the stock market and training artificial intelligence (AI) agents to identify patterns and make trading decisions.
- Three key functionalities: real-time data acquisition, AI agent simulation and training, and back-testing capabilities to evaluate strategy performance.
- The final product integrates multiple AI trading agents (SMA, RSI, RNN) along with visual representations of market data, agent performance, and back testing results.
- The overall goal was to create a platform for exploring the intersection of finance and artificial intelligence, while also gaining practical experience in application development and AI model training.

Project Structure

- Streamlit – Python library for frontend development
- Yfinance – Python library that's a wrapper for Yahoo Finance (unoffical)
- Structure of the Project:

Yfinance Explanation

- Yfinance.download(***tickers*, *start*=*None*, *end*=*None*, *actions*=*False*, *threads*=*True*, *ignore_tz*=*None*, *group_by*='*column*', *auto_adjust*=*True*, *back_adjust*=*False*, *repair*=*False*, *keepna*=*False*, *progress*=*True*, *period*='*1mo if start & end None*', *interval*='*1d*', *prepost*=*False*, *rounding*=*False*, *timeout*=*10*, *session*=*None*, *multi_level_index*=*True*)
  - data = yf.download(ticker_symbol, period="1d", interval="1m")

- Yfinance.Ticker(***ticker*, *session*=*None*)
  - ticker = yf.Ticker(ticker_symbol)

$$SMA = \frac{A_1 + A_2 + \ldots + A_n}{n}$$

Agents Descriptions (SMA)

- Simple Moving Average - The strategy works by dividing the sum of prices over N periods over N. The logic behind the equation is if the average is positive, it predicts the market is bullish. If the average is negative, it predicts the market is bearish.

Agents Descriptions (RSI)

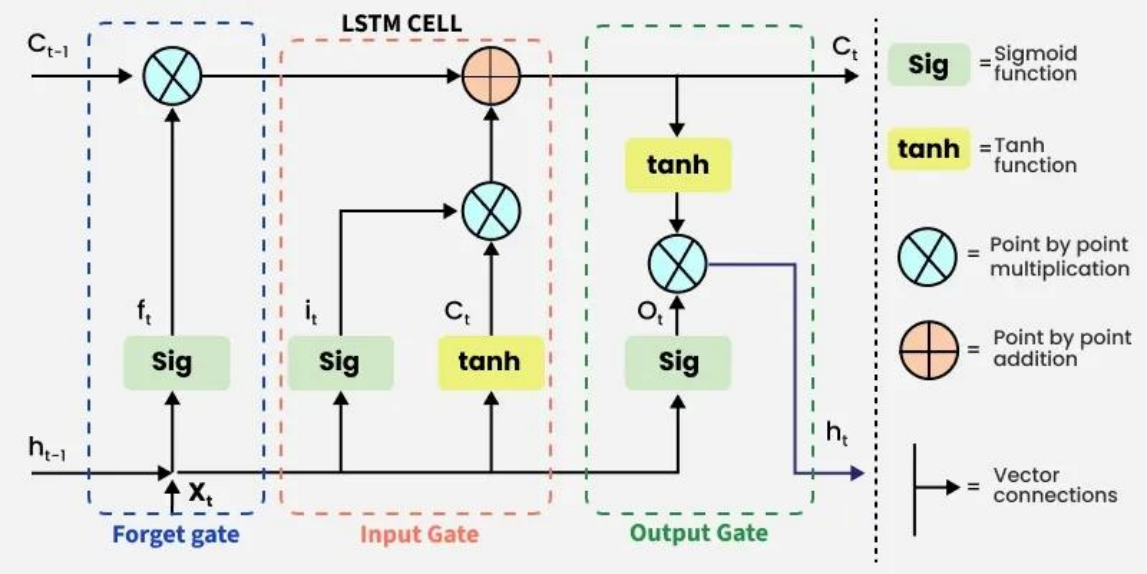
$$RS = \frac{Avg.Gain}{Avg.Loss}$$

$$RSI = 100 - \frac{100}{1 + RS}$$

- Relative Strength Index - is a momentum oscillator that measures whether a stock is overbought or oversold.
  - RSI < 30 = undersold → BUY
  - RSI > 70 = overbought → SELL
  - RSI between → HOLD

Agents Descriptions (RNN)

- Recurrent Neural Network - a machine learning approach that tries to learn patterns in sequential price data.
- LSTM (Long Short-Term Memory) - a type of recurrent neural network (RNN) designed to learn, store, and recall information over long sequences
  - Forget gate, Input gate, Output gate



Live Demo

Example Screenshots (1)

Live Trading Console

Stock Ticker

AAPL

Agent's Starting Cash ($)

10000 − +

Enable Live Updates

Live Updates by minutes

5

1 30

Reset Portfolios

Refresh RNN Model (Last 7 days)

Risk Management

Position Size (% of Cash)

20

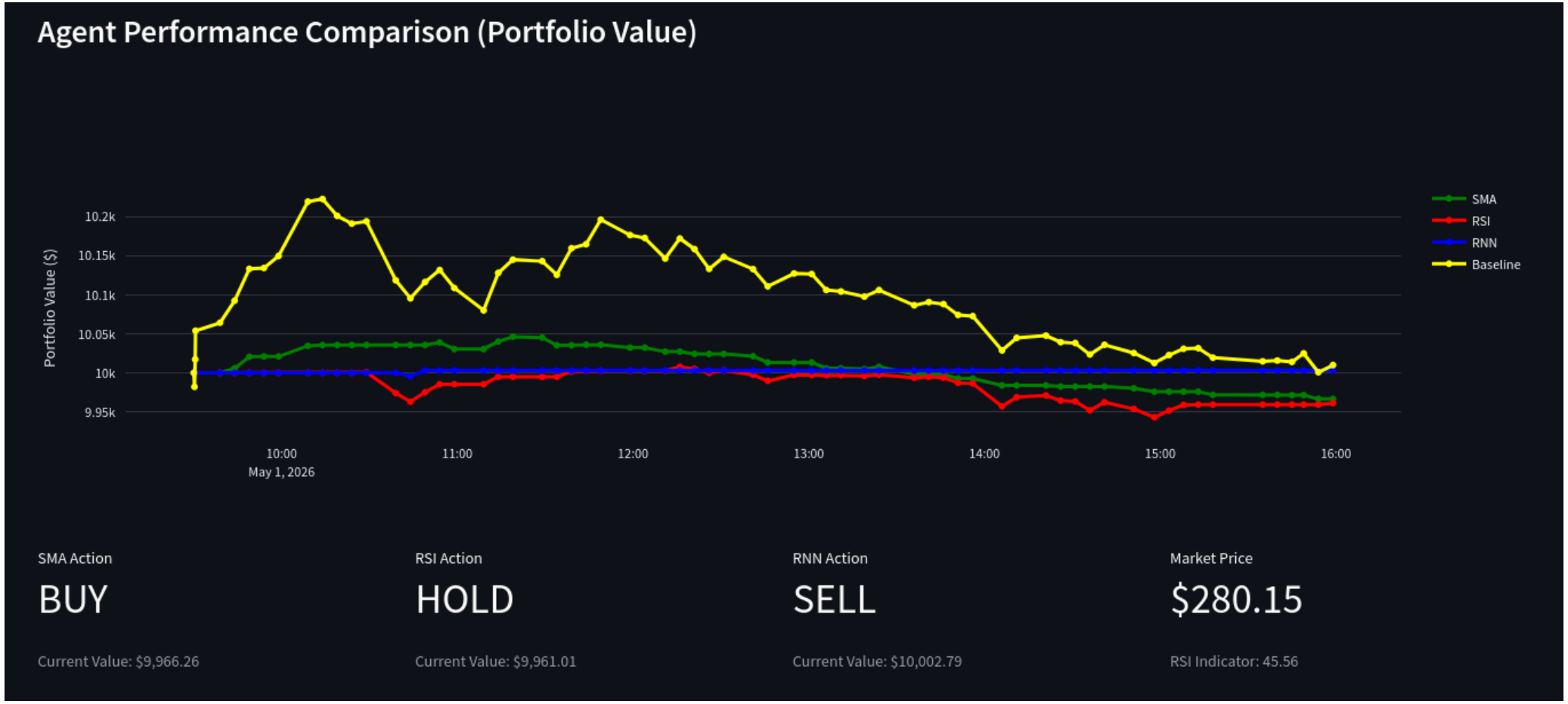
5 100

Live Market: AAPL

Agent Internal Ledgers

	Agent	Cash	Shares Owned	Total Value
0	SMA	$7,973.01	7.1149	$9,966.26
1	RSI	$7,967.36	7.1164	$9,961.01
2	RNN	$10,002.79	0.0000	$10,002.79
3	Baseline	$0.00	35.7296	$10,009.65

Example Screenshots (2)



Example Screenshots (3)

Recent Trade Activity

	Timestamp	Agent	Action	Price	Shares	Total Value
78	2026-05-01 15:59:00-04:00	SMA	BUY	280.15	7.1149	9966.2605
76	2026-05-01 15:54:00-04:00	SMA	SELL	279.895	7.1077	9966.2605
77	2026-05-01 15:54:00-04:00	RSI	BUY	279.895	7.1164	9959.1975
75	2026-05-01 15:49:00-04:00	SMA	BUY	280.57	7.1077	9971.0583
74	2026-05-01 15:45:00-04:00	SMA	SELL	280.27	7.1143	9971.0583
73	2026-05-01 15:40:00-04:00	SMA	BUY	280.32	7.1143	9971.4142
72	2026-05-01 15:18:00-04:00	SMA	SELL	280.4233	12.7915	9971.4142
70	2026-05-01 15:13:00-04:00	SMA	BUY	280.76	5.6849	9975.7211
71	2026-05-01 15:13:00-04:00	RSI	SELL	280.76	30.6932	9959.1975
69	2026-05-01 15:08:00-04:00	SMA	BUY	280.74	7.1066	9975.5788

Some Difficulties

- Plotly
- Learning finance terms
  - Ticker, Open vs Close, Baseline, Bearish vs Bullish
- Publishing Streamlit
  - Requirements.txt complications
- Training RNN
- Caching yfinance
- Getting Market Status
  - Fast_info.get() vs info.get()

```python
# 3. Train Model
def train_lstm(df):
    X, y, scaler = prepare_data(df)

    model = build_lstm_model((X.shape[1], X.shape[2]))

    model.fit(X, y, epochs=10, batch_size=32)

    # Save model + scaler
    model.save("lstm_model.h5")
    joblib.dump(scaler, "scaler.save")

    print("Model and scaler saved!")

    return model, scaler
```

```python
# FETCHING THE DATA
@st.cache_data(ttl=60)
def fetch_data(symbol, refresh_interval):
    df = yf.download(symbol, period="1d", interval="1m")
    return df

@st.cache_resource
def get_model():
    return load_lstm()
```

What I would do differently/add

More models

- **EMA (Exponential Moving Average) -** Gives more weight to recent prices, making it faster to react to price changes.
- **CNN (Convolutional Neural Networks) -** Extracting patterns from price/volume images or data windows.

Realistic features like trading delay and fees

Visually make the website more readable/customizable

- Customizable RSI ratios
- Detailed description for project on the site
- Training videos
- Pick and Choose which agents

Make own web scrapper to avoid yfinance rate limit

The End